r/learnpython • 2 yr. ago
_iam_yemisi

What's the purpose of f'strings?

Hello i am a very very beginner coder. I'm in a beginner python class and I feel like the text books just throws stuff at concepts. Why are f'strings important? If I'm understanding it right I feel like the same output could get accomplished in a different way.

143 92 Share

Join the conversation

Sort by: Best Search Comments

fiddle_n • 2y ago

F strings are just a nice way to insert variables into a template string. Yes, there are other ways - but f strings are usually the best.

What way would you use instead?

189 Reply Award Share

dididothat2019 • 2y ago

kind of like a sprintf() in C

9 Reply Award Share

_iam_yemisi OP • 2y ago

sarabooker • 2y ago

They're a clean and readable way to put variables in a string. The important part is readable, since unreadable code is bad code.

49 Reply Award Share

Bobbias • 2y ago

r/learnpython

Join

Python Education

Subreddit for posting questions and asking for general advice about all topics related to learning python.

Created Oct 2, 2009
Public

956K Members **125** Online

COMMUNITY BOOKMARKS

wiki

FAQ

R/LEARNPYTHON RULES

1. Be polite.
2. Posts to this subreddit must be requests for help learning python.
3. Replies on this subreddit must be pertinent to the question OP asked.
4. No advertising. No blogs/tutorials/videos/books/recruiting attempts.
5. No replies copy / pasted from ChatGPT or similar.

CODE HOSTING/FORMATTING

Post your code on these websites and include the link in your thread, or click on the button below to find out how to properly format code and include it in your submission text.

Reddit code formatting

Github Gist

Pastebin

repl.it

HELPFUL POSTING RESOURCES

Please check out few of these links to see how to properly ask a software development related questions.

How to ask a question

The perfect question

RELATED SUBREDDITS

r/Python
1,391,241 members

r/django
152,619 members

F-strings are fast. There's a bunch of optimizations to make them the fastest choice most of the time.

F-strings are easier to read. There's less punctuation than using multiple inputs to `print()`, and you can see exactly what's being printed where its supposed to go unlike the c style % formatting.

Also, something that hasn't really been discussed here is that you can include arbitrary code inside the curly braces inside an f-string. For example:

```
print(f"There {'are' if x > 1 else 'is'} {x} value{'s' if x > 1 else ''}.")
```

Alternately:

```
plural = x > 1 # makes more sense if this is a complex condition
print("There {'are' if plural else 'is'} {x} value{'s' if plural else ''}.")
```

Assuming x is a nonzero positive value, this detects whether x is 1 or higher and prints a grammatically correct sentence based on that. This would be much messier using either the c style % formatting or the multiple input variation of `print()`. And in the latter case, getting the spacing correct would be annoying because the spacing requirements are inconsistent between the first and third interpolation points.

This ability to embed code is great for print debugging, where you often have temporary values you want to calculate purely for the purpose of debugging.

Overall, f-strings are an extremely powerful text formatting tool with many advantages over the alternatives, and should be the default option unless there's a good reason to use something else.

40 Reply Award Share ...

RallyPointAlpha • 2y ago

Yes yes yes! Great info on fstrings!

4 Reply Award Share ...

achampi0n • 2y ago

`f-strings` are effectively syntactic sugar for `format()` command, e.g.:

```
print(f"There {'are' if x > 1 else 'is'} {x} value{'s' if x > 1 else ''}.")
```

Can be rewritten:

```
print("There {} {} value{}.".format('are' if x > 1 else 'is', x, 's' if x > 1 e
```

I find `f-string` way more readable, hence the value in this.

3 Reply Award Share ...

pfmiller0 • 2y ago

But it's not just syntactic sugar, fstrings are implemented in a different way and they are actually faster than using format.

7 Reply Award Share ...

sylfy • 2y ago

I can't quite think of any language that has gone through as many different ways to format a string as Python has. I think they finally got it right with f-strings.

2 Reply Award Share ...

interbased • 2y ago

Wow, I've been using f-strings forever now and those use cases are extremely helpful. I'm going to try those out when I can. Thanks.

1 Reply Award Share ...

cowbois • 14d ago

r/programming
6,784,010 members

r/learnprogramming
4,250,350 members

r/dailyprogrammer
238,990 members

COMMENTING GUIDELINES

- Try to guide OP to a solution instead of providing one directly.
- Provide links to related resources.
- Answer the question and highlight side-issues if any exist.
- Don't "answer and run", be prepared to respond to follow up questions.
- Proofread your answers for clarity and correctness.
- Be polite.

POSTING GUIDELINES

- Try out suggestions you get and report back.
- SSCCE Keep your code Short, Self Contained, Correct (Compilable) and provide Example
- Include the error you get when running the code, if there is one.
- Ensure your example is correct. Either the example compiles cleanly, or causes the exact error message about which you want help.
- Avoid posting a lot of code in your posts.
- Posting homework assignments is not prohibited if you show that you tried to solve it yourself.

MODERATORS

Message Mods

u/wub_wub

u/AutoModerator

u/xiongchiamiov

u/novel_yet_trivial

u/xelf

View all moderators

LANGUAGES

Português

中文 (繁體)

中文 (简体)

Français

Reddit Rules Privacy Policy User Agreement Accessibility

1 Reply Award Share ...

Bobbias • 14d ago

Those are conditional expressions going by the language documentation.

Anywhere you see `if` and `else` on the same line, they're part of an expression. The `if <condition>:` construct is the if statement.

The difference between statements and expressions is that statements are code that does not return a value, while expressions do.

Syntactically speaking, the `if` in the f-string there is the same as the expression on the right side of the assignment in this piece of code:

```
variable = 5 if a > b else 0
```

2 Reply Award Share ...

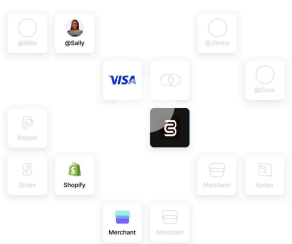
cowbois • 14d ago

Thanks!

statement: code that does not return a value
expression: code that does return a value

This subreddit is just ridiculously helpful :)

1 Reply Award Share ...



[deleted] • 2y ago

wildpantz • 2y ago

Let's say you have variables a, b and c. Let's say you want to print them out in a single statement, all 3 of them and you want to make it look pretty.

Old school way:

print("The value of a is " + a + ", the value of b is " + b + " and the value of c is " + c + ".")

F-string way:

print(f"The value of a is {a}, the value of b is {b} and the value of c is {c}.")

As you can see, not only do f string "compress" the whole statement, it's also a lot more readable. Also, there is a lot more room for mistakes in the first example (or similar techniques). Try it out yourself a few times and you will never go back to the old way!

16 Reply Award Share ...

sunohar • 2y ago

> print(f"The value of a is {a}, the value of b is {b} and the value of c is {c}.")

Shorter way

print(f"The values are {a = }, {b = } and {c = }.")

This way it prints both variable name and it's value

34 Reply Award Share ...

wildpantz • 2y ago

Bright-Profession874 • 2y ago

f strings are slower than concatenation with +(plus) though

-4 Reply Award Share ...

bohoky • 2y ago

If by "slower" you mean "more than twice as fast in typical use"

```
In [3]: %timeit "a string with two " + name + " being the first and " + name +
94.5 ns ± 0.906 ns per loop (mean ± std. dev. of 7 runs, 10,000,000 loops each)

In [4]: %timeit f"a string with two {name}  being the first and {name}  being tl
39.9 ns ± 0.185 ns per loop (mean ± std. dev. of 7 runs, 10,000,000 loops each)
```

then I agree.

However, performance concerns at the tens-of-nanoseconds level is perhaps not the most salient factor over readability.

7 Reply Award Share ...

wildpantz • 2y ago

I think it doesn't matter really unless you put them in a large loop, but I wasn't aware of this. Nice to know. Thanks

2 Reply Award Share ...

_iam_yemisi OP • 2y ago

Wow this really helps. There's no need to make separations since you can place the variable within the string. This makes so much sense thank you!

10 Reply Award Share ...

nullbyte420 • 2y ago

They just look nicer than a having a lot of %s

12 Reply Award Share ...

ThePandaBrah666 • 2y ago

f strings are particularly useful if you need to return a message using an interface such a say a chatbot or an automated email template.

For example a company has your name in a database so when they contact you they'll pull your name and do

f" Hey, {name}! Blah blah blah".

This is one of the most basic examples I can think of as I have been working in Natural Language Processing (NLP) for a while so that's all I can think of. Additionally, I had a small stint in drug discovery and I'd use fstrings to combine/add parts in a long pipeline of functions. I'd have a second variable that relied on the first and I'd do for example:

variable_1 = A0A0A1

And variable_2 = f"XXXX-{variable_1}-XXYX".

Small basic examples but yeah basically if you're not printing in terminal and you want to maintain the end result in a variable or use it further then f strings are the way to go due to their simplicity/ease of implementation.

4 Reply Award Share ...

ayyycab • 2y ago

Much cleaner way to put variables in a string than using + a dozen times. Once I got the hang of it I haven't looked back.

7 Reply Award Share ...

Miiicahhh • 2y ago • Edited 2y ago

It's just a nice and easy, user friendly way to insert variable data into a string, what I mean by this is

name = test;

f"My app name is {name}."

In comparison totext = "My app name is {}"

print(text.format(name = "test")

These essentially do the same thing but it's just way easier and more straight forward with f string.

Now coding is just kind of like that and sometimes you might find yourself trying to find meaning where there isn't any. There are some things that are a lot like added features for certain people. In comparison to a car: Some people like heated seats, some people like moon roofs, xyz. The fact is that not everyone ticks the same.

With all that being said, most things in code can be accomplished in about 1,000 different ways. So, a lot of the time the answer to: "Why would I use that if I can use this and do the same thing", would be because it's easier for you to understand. From there, sometimes the output is the same but the process of which it got that output is different. This is kind of getting down into Big O program optimization and algorithms so I wont overwhelm you with that quite yet but it gets a little clearer the more you continue to learn.

3 Reply Award Share ...

jamy1892 • 2y ago

print(f"My name is {your_name}")

print("My name is " + your_name)

Just an easier way to insert variables into strings.

edit: typo

3 Reply Award Share ...

jfp1992 • 2y ago

Instead of

"Hi" + op_username + ", " + programming_language + "uses string interpolation to make sentences more readable"

3 Reply Award Share ...

[deleted] • 2y ago

Fstrings are easy modifications of our strings so that we can insert variable values in text.

Example, "I am {age) years old."

The age variable can vary depending on the user, and we don't want to be using "if age = 0, print I am 0 years old" and so on. Hence, f strings to help us insert variables in text without having to change around too many things.

2 Reply Award Share ...

ANautyWolf • 2y ago

It's extremely convenient when you need to present data to the user or change input data to something the program likes when making it backwards compatible. It takes what could be done in many lines and brings it down to one or maybe a couple lines that are clean and easy to read.

2 Reply Award Share ...

throwaway6560192 • 2y ago

> If I'm understanding it right I feel like the same output could get accomplished in a different way.

There are multiple ways to do *anything*. The question is which of them is the best for your specific situation.

f-strings are in my opinion the best solution to the problem of inserting variables into strings.

3 Reply Award Share ...

GenericLatinPhrase • 2y ago

You'd appreciate f strings more when you're trying to debug your program and want to figure out what certain variables and functions return on specific parts of your code.

2 Reply Award Share ...

Raccoonridee • 2y ago

String formatting tools such as f-strings, `str.format()` and the older `%` syntax provide compact, readable, and often faster running code then manual workarounds such as string concatenation. The "I know I could get the same result with some sticks and duct tape" thing happens to all of us every now and then, it's just your brain resisting to learn.

There's an excellent article on string formatting that I refer to every once in a while even though I've been programming in Python since 2015: https://pyformat.info/

2 Reply Award Share ...

cybersalvy • 2y ago

Damn. Learned something awesome today. Thx for the question OP and for the feedback everyone.

2 Reply Award Share ...

_iam_yemisi OP • 2y ago

Right! The feedback has been great

1 Reply Award Share ...

Dilutant • 2y ago

It'll print out the variable name and value because of the = Lots of tricks like this with f strings

1 Reply Award Share ...

anh86 • 2y ago

Convenience! I love f-strings!

1 Reply Award Share ...

ComputerSoup • 2y ago

aside from the other uses mentioned, i'd argue f-strings are just more readable when inside a print statement. commas introduce whitespace that you may or may not want and have to adjust for, and it just breaks up the flow of reading. "text {variable} text" is very nice to follow.

1 Reply Award Share ...

Cauldron-Don-Chew • 2y ago

Forget about f strings, you should look up g strings

1 Reply Award Share ...

EnD3r8_ • 2y ago

X = 1 Print(f"x = {x}") The output would be: X = 1

1 Reply Award Share ...

Asmo__deus • 2y ago

Write code for your coworkers, not for your boss.

If you use variables with clear names and no silly abbreviations, and use f'strings, your strings will essentially read like regular human language. And that's fantastic, it's so much easier to understand.

1 Reply Award Share ...

aplarsen • 2y ago

Read PEP 498. All of the justification is there.

https://peps.python.org/pep-0498/

1 Reply Award Share ...

bahcodad • 2y ago

I'm still very much a beginner, but I'm learning that a lot of emphasis is (and should be) placed on readability, other programmers, and your future self will thank you for that.

I also believe that f-strings are relatively new and while they are currently best practice, it's important to know the older ways because you may see it used in older code.

1 Reply Award Share ...

RallyPointAlpha • 2y ago • Edited 2y ago

Dooood.

F string is my JAM! This was after years of doing it the old ways. I didn't want to change but I kept running into limitations from other methods.

It also broke my bad habbit of using math opporator + to concatenate strings.

Don't forget to wrap any non string value in str(). I also love using pp.pformat() when necessary. It's from the Pretty Print module, which I use extensively.

1 Reply Award Share ...

Potential-Tea1688 • 2y ago

1 Reply Award Share ...

dogweather • 2y ago

https://forkful.ai/en/python/interpolating-a-string/

1 Reply Award Share ...

_iam_yemisi OP • 2y ago

Wow this was a 1000x more digestible than my textbooks deadass thank you

2 Reply Award Share ...

dogweather • 2y ago

Holy sh*t, thank you. That means a lot to me. I'm writing the textbook I wish I had.

I've been working 10 hour days on the content, using it for myself too.

2 Reply Award Share ...

PhilipYip • 2y ago

```
body = 'The string to 0 is 1 2!'
```

And there are three ```str``` instances:

```
var0 = 'print'
var1 = 'hello'
var2 = 'world'
```

The objective of a formatted string is to insert these instances into the str body so a f

```
'The string to print is hello world!'
```

If the docstring of the str method format is examined:

```
body.format?
```

```
>>>Docstring:
>>>S.format(*args, **kwargs) -> str

>>>Return a formatted version of S, using substitutions from args and kwargs.
>>>The substitutions are identified by braces ('{' and '}').
>>>Type:      builtin_function_or_method
```

```
body = 'The string to {0} is {1} {2}!'
```

Syntax highlighting in most IDEs will distinguish these placeholders, making the code mor

*args represents a variable number of positional input arguments. When inserting instance

```
body.format(var0, var1, var2)
```

```
'The string to print is hello world!'
```

The str instance body can alternatively be setup to contain named variables:

```
body = 'The string to {var0_} is {var1_} {var2_}!'
```

**kwargs represents a variable number of named keyword input arguments which should match

```
body.format(var0_=var0, var1_=var1, var2_=var2)
```

```
'The string to print is hello world!'
```

The two lines above can be combined:

```
'The string to {var0_} is {var1_} {var2_}!'.format(var0_=var0, var1_=var1, var2_=var2)
```

```
'The string to print is hello world!'
```

It is more common for the placeholders to be given the same name as the instances to be i

```
'The string to {var0} is {var1} {var2}!'.format(var0=var0, var1=var1, var2=var2)
```

```
'The string to print is hello world!'
```

```
f'The string to {var0} is {var1} {var2}!'
```

'The string to print is hello world!'

If the object datamodel method __format__ is examined:

```
object.__format__?
```

>>> Signature: object.__format__(self, format_spec, /)
>>> Docstring:
>>> Default object formatter.

>>> Return str(self) if format_spec is empty. Raise TypeError otherwise.
>>> Type: method_descriptor

Supposing a str instance is instantiated:

```
greeting = 'Hello World!'
```

The format specification for a str instance has the form:

'0ns' where n is an integer, s means str and 0 is used to fill in blank spaces.

```
greeting.__format__('s')
```

'Hello World!'

```
greeting.__format__('22s')
```

'Hello World!   '

```
greeting.__format__('022s')
```

'Hello World!0000000000'

The formatter specifier options differ for each datatype. Normally a colon is used to inc

```
f'The string to {var0:s} is {var1} {var2}!'
```

'The string to print is hello world!'

The str format specifier can specify an integer number of characters:

```
f'The string to {var0:10s} is {var1} {var2}!'
```

'The string to print   is hello world!'

If prefixed with 0 then trailing spaces will be displayed using 0:

```
f'The string to {var0:010s} is {var1:s} {var2:s}!'
```

'The string to print00000 is hello world!'

In the above str instances were inserted into a str instance body. It is more common to i

num2 = 0.0000123456789
num3 = 12.3456789
```

```
f'The numbers are {num1}, {num2} and {num3}.'
```

'The numbers are 1, 1.23456789e-05 and 12.3456789.'

The format specifier for an integer decimal (d) can be used:

```
f'The numbers are {num1:d}, {num2} and {num3}.'
```

'The numbers are 1, 1.23456789e-05 and 12.3456789.'

```
f'The numbers are {num1:5d}, {num2} and {num3}.'
```

'The numbers are   1, 1.23456789e-05 and 12.3456789.'

```
f'The numbers are {num1:05d}, {num2} and {num3}.'
```

'The numbers are 00001, 1.23456789e-05 and 12.3456789.'

```
f'The numbers are {num1: 05d}, {num2} and {num3}.'
```

'The numbers are 0001, 1.23456789e-05 and 12.3456789.'

Again the number of characters in the string the number should occupy can be specified. l

Notice one of the five characters is a space because a space is part of the formatter spe

```
f'The numbers are {num1}, {num2:g} and {num3:g}.'
```

'The numbers are 1, 1.23457e-05 and 12.3457.'

The e can be used for float exponential format:

```
f'The numbers are {num1}, {num2:e} and {num3:e}.'
```

'The numbers are 1, 1.234568e-05 and 1.234568e+01.'

The number of places after the decimal point can be specified:

```
f'The numbers are {num1}, {num2:0.3e} and {num3:0.3e}.'
```

'The numbers are 1, 1.235e-05 and 1.235e+01.'

A fixed format can also be used:

```
f'The numbers are {num1}, {num2:f} and {num3:f}.'
```

'The numbers are 1, 0.000012 and 12.345679.'

Once again the number of spaces after the decimal point can be specified:

```
```
```

```
'The numbers are 1, 0.000 and 12.346.'
```

float instances can use the general (g), exponential (e) and fixed (f) format specifiers.

0 Reply Award Share ...

In short: They are easier to read.

Do you know how translations are working (e.g. GNU gettext)?

A translator without any technical knowledge will see this string:

```
"It is currently {} degrees"
```

because in your Python code it is

```
print("It is currently {} degrees".format(temperature))
```

Here the translator don't get enough context information to provide a good translation. But if it is

```
"It is currently {temperature} degrees"
```

the translator can be sure what the meaning is and provide a correct translation.

0 Reply Award Share ...

roywill2 • 2y ago

I dont like fstrings all those spurious {}. I like to separate presentation from content:

```
s = '%d buns cost %4.2f'
s = s % (nbuns, cost)
```

0 Reply Award Share ...